

Adaptive Retrieval for Code Completion

Project status — RQ1 baselines complete; cascade pipeline ready

1. Goal

We study whether **adaptive retrieval** — selectively triggering RAG only on instances where it would help — outperforms always-retrieve and never-retrieve baselines for repository-level code completion. Our novel contribution is adding a **static-analysis cascade stage** to the CARD framework (Zhang et al. 2024). The hypothesis: CARD's logit-based uncertainty signal misses a class of failures — confidently-generated identifiers that don't actually exist in the repository — and static analysis of the prediction's AST can detect these directly. The cascade is asymmetric by design: static analysis can only *add* retrievals to CARD's decisions, never remove them.

2. Research questions

RQ	Question	Hypothesis
RQ1	How does the proposed cascade perform against always-retrieve and never-retrieve baselines on CrossCodeEval-Python, by accuracy (EM, ES, Identifier-F1) and efficiency (% retrieval, latency)?	H1. The cascade matches or beats always-retrieve on accuracy while performing fewer retrievals.
RQ2	Does the static-analysis stage reduce the rate of identifier hallucinations versus vanilla CARD on the same instances?	H2. The cascade cuts per-instance hallucination rate by at least 20% relative to vanilla CARD, significant under McNemar's exact test ($p < 0.05$).

3. Architecture

Three asymmetric stages: zero-shot generation, CARD's uncertainty gate, and a static-analysis gate that runs only when CARD says skip. Boxes in blue are reimplemented from CARD; the orange box is our contribution.

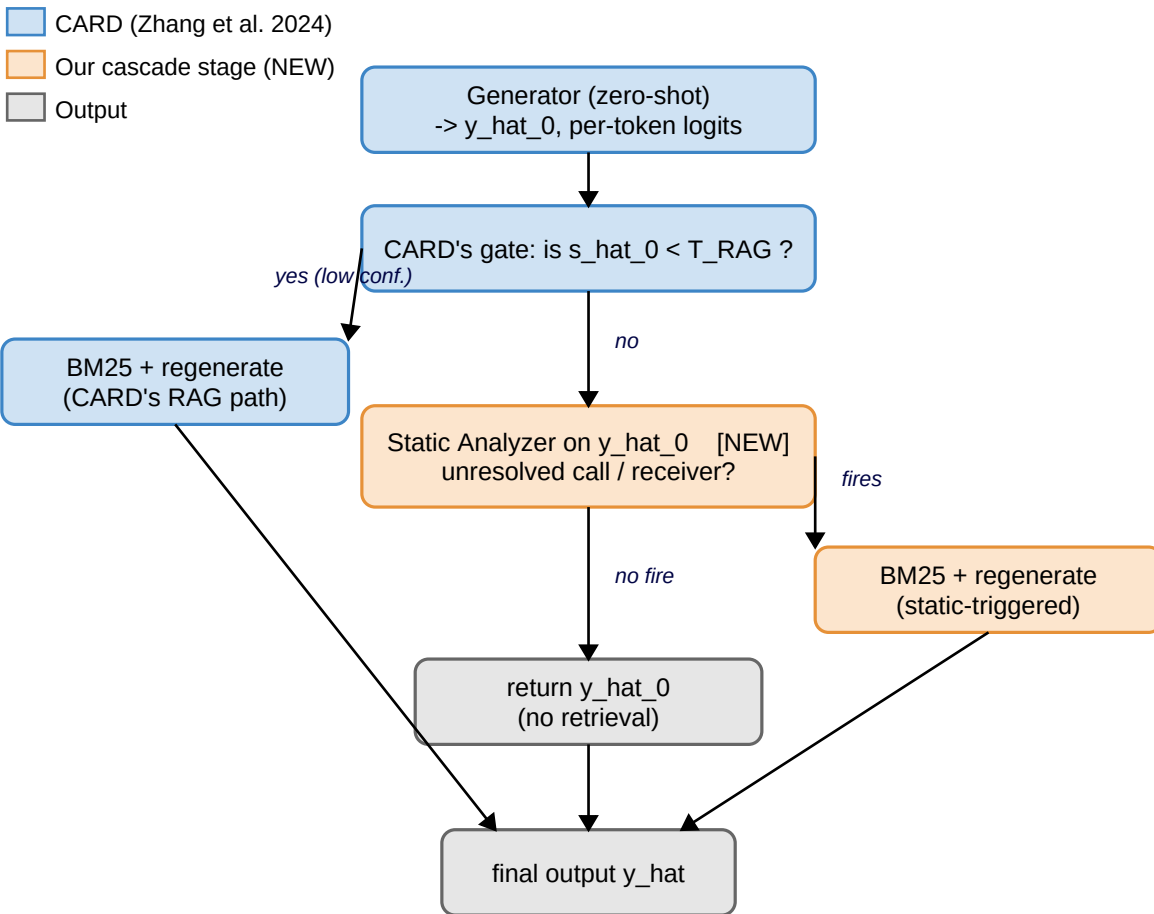


Figure 1. The cascade pipeline. CARD gate fires on low-confidence predictions; static-analysis gate fires when y_hat_0 references a call target or attribute receiver that doesn't resolve in the current scope, builtins, or the repository symbol table. Either trigger leads to a BM25 + regenerate step; otherwise y_hat_0 is returned without retrieval.

4. Implementation

All five infrastructure phases are complete (~3000 lines of Python across 14 modules, 194 passing tests, ruff clean). The Generator is abstracted behind a single interface with three real backends (HuggingFace transformers, vLLM, and mlx-lm for Apple Silicon) plus a Mock for tests, all returning the same per-token probability and entropy data CARD's features need.

- **CARD components** (blue in Figure 1) — the 13-D feature vector from per-token logits, a LightGBM Estimator that regresses edit similarity on those features, and the gate that fires retrieval when $s_hat_0 < T_RAG$.
- **Cascade components** (orange) — a tree-sitter-based static analyzer that parses each prediction's AST, classifies every structurally-significant identifier (call targets, attribute receivers, subscript values) as in-file / cross-file / unresolved, and fires when any such identifier fails to resolve.
- **Shared infrastructure** — BM25 retriever, FIM prompt assembly, metrics (EM, ES, Identifier-F1, repo-symbol-precision, hallucination flag, McNemar test, paired bootstrap), per-instance JSONL writer, and a disk-backed generation cache that means the same zero-shot prompt is computed once and reused across configs.
- **Evaluation** — six configurations (C1 no-retrieve, C2 always-retrieve, C3 CARD, C4 cascade, C5 static-only ablation, C6 oracle) and the orchestrator that runs them on CrossCodeEval-Python (~2,665 instances).

5. Baseline results

Both baselines have been run end-to-end on the full CrossCodeEval-Python (2,665 instances) with Qwen2.5-Coder-1.5B via mlx-lm on M4 Pro.

Config	Retr. %	EM	ES	IdF1	RSP	Hall.% (strict)	Lat. (ms)
C1 no-retrieve	0.0	0.08	0.27	0.46	0.54	0.69	1,255
C2 always-retrieve	100.0	0.34	0.47	0.62	0.54	0.77	2,127

RSP = repository symbol precision (fraction of identifiers in a prediction that resolve). Hall.% = at least one structurally significant identifier doesn't resolve, computed against the per-repository chunk union.

Key observations

- **Retrieval is consequential on CCE-Python** — EM jumps 4.25× (0.08 to 0.34), ES rises 74% (0.27 to 0.47). The benchmark was curated for cross-file dependence, and that shows.
- **Cost is a 70% latency penalty** on every instance (1.25 s to 2.13 s). This is the budget the cascade has to trade against.
- **Hallucination rates are high in both configs**. Even with retrieval, 77% of predictions contain at least one unresolved identifier in a structurally significant position. Retrieval fixes the *targeted* identifier (the one that drives EM/ES) but doesn't systematically reduce the rate of *peripheral* hallucinated names.

6. Next steps

- **Decide how to measure hallucinations.** The current hallucination rate (~77%) is high because our symbol table per instance is small (chunks selected by BM25 from a real repo) and even good predictions naturally reference names outside those chunks. Several refinements are under consideration: import-following resolution, confidence-weighted filtering, or replacing the binary flag with a continuous metric (mean unresolved-identifier count per prediction). The choice affects both the absolute number reportable in the paper and the cascade's trigger rate.
- **Calibrate the Estimator on the 7B model.** CARD's Estimator is generator-specific; once we scale up the underlying code LM, the (features, ES) training-data step must be repeated. With a 7B model on GPU this is ~hours on an A100; alternatives are running it overnight locally via mlx-lm or scoping a smaller Estimator training set with a documented tradeoff in MSE.
- Once both are settled, C3 (CARD) and C4 (cascade) can be run to close out RQ1 and answer RQ2.